Cryptography
Practical 4: Block Ciphers: DES and AES

Martin Harrigan, ✉ martin.harrigan@setu.ie

In this practical we will use Python's `des` and `pyca/cryptography` packages to experiment with block ciphers and modes of operation. You should familiarise yourself with the API documentation for the `pyca/cryptography` package at ↗ `https://cryptography.io`. In particular, the documentation for block ciphers and modes of operation is available under The Hazardous Materials Layer ▹ Primitives ▹ Symmetric Encryption.

1 The Data Encryption Standard (DES)

DES is a symmetric-key block cipher published by the National Institute of Standards and Technology (NIST) and based on the Feistel Scheme. It uses a 64 bit block, 16 rounds, and has an *effective* 56 bit security.

Many cryptography libraries have retired their implementations of DES and Triple DES (3DES), e.g., OpenSSL 3 moved DES to its legacy provider, which is not loaded by default, and `pyca/cryptography` moved 3DES to its `hazmat.decrepit` module. You should avoid any such implementations where possible.

Section A provides source code that encrypts and decrypts using DES and is based on a Python implementation of the block cipher (↗ `https://pypi.org/project/des/`). The code creates two keys, `key_short_des` and `key_long_des`. The first is a 64 bit key, i.e., an effective 56 bit key, as required by DES. The second is a 128 bit key, i.e., an effective 112 bit key, as required by two-key 3DES.

Experiment with various plaintexts and keys. What are the restrictions on their lengths?

If you want to handle plaintext messages of arbitrary length, you need to use padding:

```
ciphertext_bytes = key_des.encrypt(plaintext_bytes, padding=True)
...
plaintext_bytes_2 = key_des.decrypt(ciphertext_bytes, padding=True)
```

If you want to use *CBC mode* instead of *ECB mode* you need to set an `initial` value or *initialisation vector*:

```
ciphertext_bytes = key_des.encrypt(plaintext_bytes, initial=0, padding=True)
...
plaintext_bytes_2 = key_des.decrypt(ciphertext_bytes, initial=0, padding=True)
```

The following ciphertext in hexadecimal notation was produced using the DES algorithm in CBC mode (`initial=0`) with padding (`padding=True`) and the key `superkey`:

```
37 63 03 54 1f c9 ad dd
cc 15 60 54 c7 33 2e 8e
ea a0 86 e6 1e 6a 8e f7
a6 15 b8 cf 1f 9c 52 05
a9 6a 31 1f fd ad 15 66
```

What is the corresponding plaintext?

2 The Advanced Encryption Standard (AES)

AES is a symmetric-key block cipher published by NIST and based on a Substitution-Permutation Network. It uses a 128 bit block, 10/12/14 rounds, and 128/192/256 bit keys, respectively.

Section B provides source code that encrypts and decrypts using AES and is based on a Python package that includes both high-level "recipes" and low-level interfaces to common cryptographic algorithms (↗ `https://pypi.org/project/cryptography/`).

Experiment with various plaintexts and keys. What are the restrictions on their lengths?
If you want to handle plaintext messages of arbitrary length, you need to use padding:

```
from cryptography.hazmat.primitives import padding
...
padder = padding.PKCS7(algorithms.AES.block_size).padder()
unpadder = padding.PKCS7(algorithms.AES.block_size).unpadder()
...
padded_bytes = padder.update(plaintext_bytes) + padder.finalize()
ciphertext_bytes = aes_encryptor.update(padded_bytes)
...
padded_bytes_2 = aes_decryptor.update(ciphertext_bytes)
plaintext_bytes_2 = unpadder.update(padded_bytes_2) + unpadder.finalize()
```

If you want to use *CBC mode* instead of *ECB mode* you need to change the mode and set an *initialisation vector*:

```
aes_cipher = Cipher(algorithms.AES(key_bytes),
                    modes.CBC(bytearray(16)))
```

The following ciphertext in hexadecimal notation was produced using the AES algorithm in CBC mode (`iv` = all zeros) with padding and the key `thisisonelongkey`:

```
39 bd 6a 4b bb 32 17 33
ca da 23 40 fb cb f8 a5
c9 d8 df 6b de 0a 25 96
9d 7b 98 13 21 9a ac 0c
4d 4e b6 52 0b e8 72 71
d3 19 84 11 d3 a3 ec c8
```

What is the corresponding plaintext?

2.1 A Note on Fernet

Fernet (pronounced /fə(ɹ)'nɛt/) is a high-level recipe in Python's `pyca/cryptography` package. It performs symmetric encryption and decryption using AES-128 in CBC mode. It tries to standardise the format of the key and the format of the ciphertext. The formats are described at `https://github.com/fernet/spec/blob/master/Spec.md`. However, the specification has not seen widespread adoption. You should *not* use this recipe in our module. Instead, we will use the low-level primitives.

3 Known Answer Tests (KATs)

NIST publishes a list of Known Answer Tests (KATs) that can be used to validate an implementation and configuration of AES. Download the AES KATs from: `http://csrc.nist.gov/groups/STM/cavp/documents/aes/KAT_AES.zip`.

Unzip the file. You will find a collection of Response (RSP) files. You can open them in a text editor. For example, `ECBGFSbox128.rsp` contains:

```
# CAVS 11.1
# Config info for aes_values
# AESVS GFSbox test data for ECB
# State : Encrypt and Decrypt
# Key Length : 128
# Generated on Fri Apr 22 15:11:26 2011

[ENCRYPT]

COUNT = 0
KEY = 00000000000000000000000000000000
PLAINTEXT = f34481ec3cc627bacd5dc3fb08f273e6
CIPHERTEXT = 0336763e966d92595a567cc9ce537f5e
```

```
...

COUNT = 6
KEY = 00000000000000000000000000000000
PLAINTEXT = 58c8e00b2631686d54eab84b91f0aca1
CIPHERTEXT = 08a4e2efec8a8e3312ca7460b9040bbf
```

It contains a list of test vectors. Each test vector contains a `KEY`, `PLAINTEXT` and `CIPHERTEXT`. If you encrypt the `PLAINTEXT` with the `KEY` using AES in ECB mode, you should get the corresponding `CIPHERTEXT`. The files for CBC, CFB and OFB modes also contain initialisation vectors (`IVs`).

Use your code from Section 2 to validate each of the following four test vectors.

`ECBGFSbox128.rsp` (ECB mode, 128 bit key):

```
COUNT = 0
KEY = 00000000000000000000000000000000
PLAINTEXT = f34481ec3cc627bacd5dc3fb08f273e6
CIPHERTEXT = 0336763e966d92595a567cc9ce537f5e
```

`CBCKeySbox192.rsp` (CBC mode, 192 bit key):

```
COUNT = 0
KEY = e9f065d7c13573587f7875357dfbb16c53489f6a4bd0f7cd
IV = 00000000000000000000000000000000
PLAINTEXT = 00000000000000000000000000000000
CIPHERTEXT = 0956259c9cd5cfd0181cca53380cde06
```

`CFB128VarKey256.rsp` (CFB mode, 256 bit key):

```
COUNT = 0
KEY = 8000000000000000000000000000000000000000000000000000000000000000
IV = 00000000000000000000000000000000
PLAINTEXT = 00000000000000000000000000000000
CIPHERTEXT = e35a6dcb19b201a01ebcfa8aa22b5759
```

`OFBVarKey256.rsp` (OFB mode, 256 bit key):

```
COUNT = 0
KEY = 8000000000000000000000000000000000000000000000000000000000000000
IV = 00000000000000000000000000000000
PLAINTEXT = 00000000000000000000000000000000
CIPHERTEXT = e35a6dcb19b201a01ebcfa8aa22b5759
```

4 Multiblock Message Tests (MMTs)

NIST also publishes test vectors that span multiple blocks. These are known as Multiblock Message Tests (MMTs). Download the AES MMTs from: `http://csrc.nist.gov/groups/STM/cavp/documents/aes/aesmmt.zip`.

Unzip the file. You will find a collection of Response (RSP) files. You can open them in a text editor. For example, `ECBMMT128.rsp` contains:

```
# CAVS 11.1
# Config info for aes_values
# AESVS MMT test data for ECB
# State : Encrypt and Decrypt
# Key Length : 128
# Generated on Fri Apr 22 15:11:26 2011

[ENCRYPT]

COUNT = 0
KEY = edfdb257cb37cdf182c5455b0c0efebb
```

```
PLAINTEXT = 1695fe475421cace3557daca01f445ff
CIPHERTEXT = 7888beae6e7a426332a7eaa2f808e637

...

COUNT = 9
KEY = ebea9c6a82213a00ac1d22faea22116f
PLAINTEXT = 451f45663b44fd005f3c288ae57b383883f02d9ad3dc1715...
CIPHERTEXT = 01043053f832ef9b911ed387ba577451e30d51d4b6b11f31...
```

Use your code from Section 2 to validate the following test vector:
`CBCMMT256.rsp` (CBC mode, 256 bit key):

```
COUNT = 9
KEY = 48be597e632c16772324c8d3fa1d9c5a9ecd010f14ec5d110d3bfec376c5532b
IV = d6d581b8cf04ebd3b6eaa1b53f047ee1
PLAINTEXT =
0c63d413d3864570e70bb6618bf8a4b9585586688c32bba0a5ecc1362fada74a
da32c52acfd1aa7444ba567b4e7daaecf7cc1cb29182af164ae5232b00286869
5635599807a9a7f07a1f137e97b1e1c9dabc89b6a5e4afa9db5855edaa575056
a8f4f8242216242bb0c256310d9d329826ac353d715fa39f80cec144d6424558
f9f70b98c920096e0f2c855d594885a00625880e9dfb734163cecef72cf030b8
CIPHERTEXT =
fc5873e50de8faf4c6b84ba707b0854e9db9ab2e9f7d707fbba338c6843a18fc
6facebaf663d26296fb329b4d26f18494c79e09e779647f9bafa87489630d79f
4301610c2300c19dbf3148b7cac8c4f4944102754f332e92b6f7c5e75bc6179e
b877a078d4719009021744c14f13fd2a55a2b9c44d18000685a845a4f632c7c5
6a77306efa66a24d05d088dcd7c13fe24fc447275965db9e4d37fbc9304448cd
```

Appendices

A Python Code for Encrypting and Decrypting Using DES

```
from des import DesKey

# DES (Data Encryption Standard). This uses the `des` library (not `cryptography`).
# DES uses 56-bit keys and is considered insecure; 3DES uses two or three keys.

key_short = "apoorkey"
key_short_des = DesKey(bytes(key_short, "utf-8"))

print("Short Key: " + key_short)
print("Short Key Single?: " + str(key_short_des.is_single())) # 8 bytes = single DES
print("Short Key Triple?: " + str(key_short_des.is_triple()))

key_long = "thebestsecretkey" # 16 bytes = triple DES (two-key)
key_long_des = DesKey(bytes(key_long, "utf-8"))

print("Long Key: " + key_long)
print("Long Key Single?: " + str(key_long_des.is_single()))
print("Long Key Triple?: " + str(key_long_des.is_triple()))

key_des = key_short_des

plaintext = "thebestplaintext"
plaintext_bytes = bytes(plaintext, "utf-8")
print("Plaintext: " + plaintext)

ciphertext_bytes = key_des.encrypt(plaintext_bytes)
ciphertext = ciphertext_bytes.hex()
print("Ciphertext: " + ciphertext)

plaintext_bytes_2 = key_des.decrypt(ciphertext_bytes)
plaintext_2 = str(plaintext_bytes_2, "utf-8")
print("Original Plaintext: " + plaintext_2)
```

Listing 1: des_encrypt_decrypt.py

B Python Code for Encrypting and Decrypting Using AES

```
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes

# AES in ECB mode. Each block is encrypted independently (no IV).
# Note: ECB is insecure for real use because identical plaintext blocks
# produce identical ciphertext blocks, revealing patterns in the data.

key = "thebestsecretkey" # 16 bytes = AES-128
key_bytes = bytes(key, "utf-8")
print("Key: " + key)

aes_cipher = Cipher(algorithms.AES(key_bytes), modes.ECB())
aes_encryptor = aes_cipher.encryptor()
aes_decryptor = aes_cipher.decryptor()

# Plaintext must be a multiple of the block size (16 bytes) in ECB mode
plaintext = "thebestplaintext"
plaintext_bytes = bytes(plaintext, "utf-8")
print("Plaintext: " + plaintext)

ciphertext_bytes = aes_encryptor.update(plaintext_bytes) + aes_encryptor.finalize()
ciphertext = ciphertext_bytes.hex()
print("Ciphertext: " + ciphertext)

plaintext_bytes_2 = aes_decryptor.update(ciphertext_bytes) + aes_decryptor.finalize()
plaintext_2 = str(plaintext_bytes_2, "utf-8")
print("Original Plaintext: " + plaintext_2)
```

Listing 2: aes_encrypt_decrypt.py